

Chapter 4

Data Acquisition Features: Web APIs and Scraping

Data analysis often works with data from numerous sources, including databases, web services, and files prepared by other applications. In this chapter, you will be guided through two projects to add additional data sources to the baseline application from the previous chapter. These new sources include a web service query, and scraping data from a web page.

This chapter's projects cover the following essential skills:

- Using the **requests** package for Web API integration. We'll look at the Kaggle API, which requires signing up to create an API token.
- Using the **Beautiful Soup** package to parse an HTML web page.
- Adding features to an existing application and extending the test suite to cover these new alternative data sources.

It's important to recognize this application has a narrow focus on data acquisition. In later chapters, we'll validate the data and convert it to a more useful form. This reflects a separation of the following distinct concerns:

- Downloading and extracting data from the source are the foci of this chapter and the next.
- Inspection begins in [*Chapter 6, Project 2.1: Data Inspection Notebook*](#).
- Validating and cleaning the data starts in [*Chapter 9, Project 3.1: Data Cleaning Base Application*](#).

Each stage in the processing pipeline is allocated to separate projects. For more background, see [*Chapter 2, Overview of the Projects*](#).

We'll start by looking at getting data using an API and a RESTful web service. This will focus on the Kaggle site, which means you will need to sign up with Kaggle to get your own, unique API key. The second project will scrape HTML content from a website that doesn't offer a useful API.

4.1 Project 1.2: Acquire data from a web service

It's common to need data provided by a Web API. One common design approach for web services is called RESTful; it's based on a number of concepts related to using the HTTP protocol to transfer a representation of an object's state.

For more information on RESTful services, see *Building RESTful Python Web Services* (<https://www.packtpub.com/product/building-restful-python-web-services/9781786462251>).

A RESTful service generally involves using the HTTP protocol to respond to requests from client applications. The spectrum of request types includes a number of verbs like get, post, put, patch, and delete. In many cases, the service responds with a JSON document. It's also possible to receive a file that's a stream of NDJSON documents, or even a file that's a ZIP archive of data.

We'll start with a description of the application, and then move on to talk about the architectural approach. This will be followed with a detailed list of deliverables.

4.1.1 Description

Analysts and decision-makers need to acquire data for further analysis. In this case, the data is available from a RESTful web service. One of the most fun small data sets to work with is Anscombe's Quartet – <https://www.kaggle.com/datasets/carlmcbrideellis/data-anscombes-quartet>

Parts of this application are an extension to the project in [Chapter 9, Project 3.1: Data Cleaning Base Application](#). The essential behavior of this application will be similar to the previous project. This project will use a CLI application to grab data from a source.

The **User Experience (UX)** will also be a command-line application with options to fine-tune the data being gathered. Our expected command line should look something like the following:

```
% python src/acquire.py -o quartet -k ~/Downloads/kaggle.json \
--zip carlmcbrideellis/data-anscombes-quartet
```

The `-o quartet` argument specifies a directory into which four results are written. These will have names like `quartet/series_1.json`.

The `-k kaggle.json` argument is the name of a file with the username and Kaggle API token. This file is kept separate from the application software. In the example, the file was in the author's `Downloads` folder.

The `--zip` argument provides the "reference" — the owner and data set name — to open and extract. This information is found by examining the details of the Kaggle interface.

An additional feature is to get a filtered list of Kaggle data sets. This should be a separate `--search` operation that can be bundled into a single application program.

```
% python src/acquire.py --search -k ~/Downloads/kaggle.json
```

This will apply some search criteria to emit a list of data sets that match the requirements. The lists tend to be quite large, so this needs to be used with care.

The credentials in the file are used to make the Kaggle API request. In the next sections, we'll look at the Kaggle API in general. After that, we'll look at the specific requests required to locate the reference to the target data set.

The Kaggle API

See <https://www.kaggle.com/docs/api> for information on the Kaggle API. This document describes some command-line code (in Python) that uses the API.

The technical details of the RESTful API requests are at <https://github.com/Kaggle/kaggle-api/blob/master/KaggleSwagger.yaml>. This document describes the requests and responses from the Kaggle API server.

To make use of the RESTful API or the command-line applications, you should register with Kaggle. First, sign up with `Kaggle.com`. Next, navigate to the public profile page. On this page, there's an API section. This section has the buttons you will use to generate a unique API token for your registered username.

The third step is to click the **Create New Token** button to create the token file. This will download a small JSON file with your registered username and unique key. These credentials are required by the Kaggle REST API.

The ownership of this file can be changed to read-only by the owner. In Linux and macOS, this is done with the following command:

```
% chmod 400 ~/Downloads/kaggle.json
```

Do not move the Kaggle credentials file named `kaggle.json` into a directory where your code is also located. It's tempting, but it's a terrible security mistake because the file could get saved to a code repository and become visible to anyone browsing your code. In some enterprises, posting keys in code repositories — even internal repositories — is a security lapse and a good reason for an employee to be terminated.

Because Git keeps a very complete history, it's challenging to remove a commit that contains keys.

Keep the credentials file separate from your code.

It's also a good idea to add `kaggle.json` to a `.gitignore` file to make extra sure that it won't be uploaded as part of a commit and push.

About the source data

This project will explore two separate kinds of source data. Both sources have the same base path of <https://www.kaggle.com/api/v1/>. Trying to query this base path won't provide a useful response; it's only the starting point for the paths that are built to locate specific resources.

- JSON documents with summaries of data sets or metadata about data sets. These come from appending `datasets/list` to the base path.
- A ZIP archive that contains the data we'll use as an example. This comes from appending `datasets/download/{ownerSlug}/{datasetSlug}` to the base path. The `ownerSlug` value is "carlmcbrideellis". The `datasetSlug` value is "data-anscombes-quartet". A given data set has a `ref` value as a reference string with the required "ownerSlug/datasetSlug" format.

The JSON documents require a function to extract a few relevant fields like `title`, `ref`, `url`, and `totalBytes`. This subset of the available metadata can make it easier to locate useful, interesting data sets. There are numerous other properties available for search, like `usabilityRating`; these attributes can distinguish good data sets from experiments or classroom work.

The suggested data set — the Anscombe Quartet — is available as a ZIP-compressed archive with a single item inside it. This means the application must handle ZIP archives and expand a file contained within the archive. Python offers the `zipfile` package to handle locating the CSV file within the archive. Once this file is found, the existing programming from the previous chapter ([Chapter 3. Project 1.1: Data Acquisition Base Application](#)) can be used.

There are thousands of Kaggle data sets. We'll suggest some alternatives to the Anscombe Quartet in the [Extras](#).

This section has looked at the input, processing, and output of this application. In the next section, we'll look at the overall architecture of the software.

4.1.2 Approach

We'll take some guidance from the C4 model (<https://c4model.com>) when looking at our approach:

- **Context:** For this project, a context diagram would show a user extracting data from a source. You may find it helpful to draw this diagram.
- **Containers:** One container is the user's personal computer. The other container is the Kaggle website, which provides the data.
- **Components:** We'll address the components below.
- **Code:** We'll touch on this to provide some suggested directions.

It's important to consider this application as an extension to the project in [Chapter 3. Project 1.1: Data Acquisition Base Application](#). The base level of architectural design is provided in that chapter.

In this project, we'll be adding a new `kaggle_client` module to download the data. The overall application in the `acquire` module will change to make use of this new module. The other modules should remain unchanged.

The legacy component diagram is shown in [Figure 4.1](#).

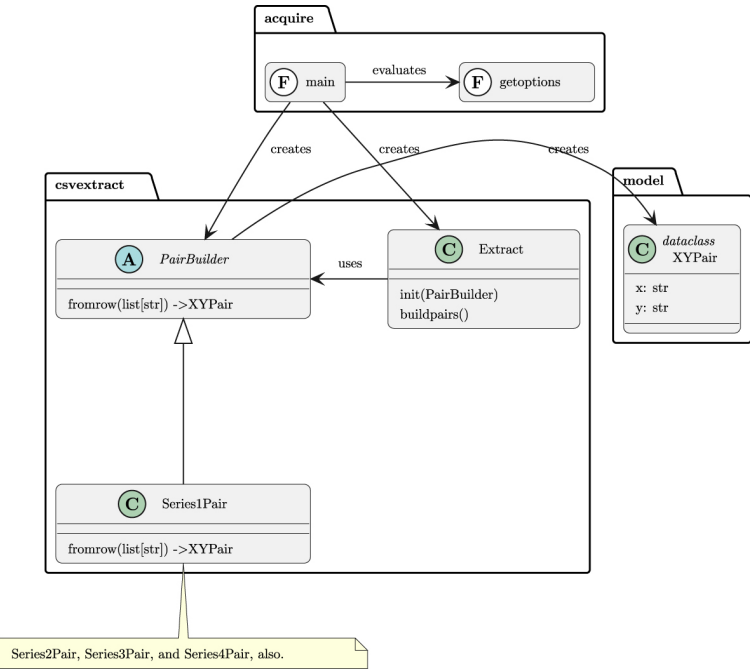


Figure 4.1: Legacy Components

A new architecture can handle both the examination of the JSON data set listing, as well as the download of a single ZIP file. This is shown in [Figure 4.2](#).

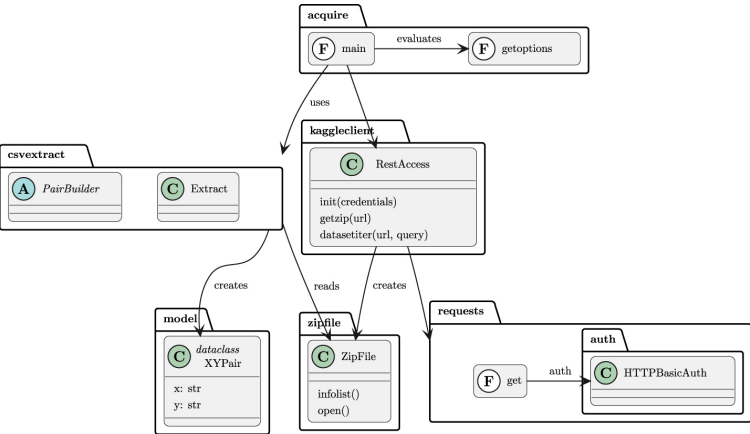


Figure 4.2: Revised Component Design

The new module here is the `kaggle_client` module. This has a class, `RestAccess`, that provides methods to access Kaggle data. It can reach into the Kaggle data set collection and retrieve a desired ZIP file. Additional methods can be added to examine the list of data sets or get data set metadata.

The `RestAccess` class is initialized with the contents of the `kaggle.json` file. As part of initialization, it can create the required authentication object for use in all subsequent calls.

In the following sections, we'll look at these features of the `RestAccess` class:

- Making API requests in general.
- Getting the ZIP archive.
- Getting the list of data sets.
- Handling the rate-limiting response.

We'll start with the most important feature, making API requests in a general way.

Making API requests

The component diagram shows the `requests` package as the preferred way to access RESTful APIs. This package should be added to the project's `pyproject.toml` and installed as part of the project's virtual environment.

It's also sensible to make RESTful API requests with the `urllib` package. This is part of the standard library. It works nicely and requires no additional installation. The code can become rather complicated-looking, however, so it's not as highly recommended as the `requests` package.

The essential benefit of using `requests` is creating an authentication object and providing it in each request. We often use code like the following example:

```
import json
from pathlib import Path
import requests.auth

keypath = Path.home() / "Downloads" / "kaggle.json"
with keypath.open() as keyfile:
    credentials = json.load(keyfile)
    auth = requests.auth.HTTPBasicAuth(
        credentials['username'], credentials['key']
    )
```

This can be part of the `__init__()` method of the `RestAccess` class.

The `auth` object created here can be used to make all subsequent requests. This will provide the necessary username and API token to validate the user. This means other methods can, for example, use `requests.get()` with a keyword parameter value of `auth=self.auth`. This will correctly build the needed `Authorization` headers in each request.

Once the class is initialized properly, we can look at the method for downloading a ZIP archive

Downloading a ZIP archive

The `RestAccess` class needs a `get_zip()` method to download the ZIP file. The parameter is the URL for downloading the requested data set.

The best approach to building this URL for this data set is to combine three strings:

- The base address for the APIs, `https://www.kaggle.com/api/v1`.
- The path for downloads, `/datasets/download/`.
- The reference is a string with the form: `{ownerSlug}/{datasetSlug}`.

This is an ideal place for a Python f-string to replace the reference in the URL pattern.

The output from the `get_zip()` method should be a `Path` object. In some cases, the ZIP archives are gigantic and can't be processed entirely in memory. In these extreme cases, a more complicated, chunked download is required. For these smaller files used by this project, the download can be handled entirely in memory. Once the ZIP file has been written, the client of this `RestAccess` class can then open it and extract the useful member.

A separate client function or class will process the content of the ZIP archive file. The following is **not** part of the `RestAccess` class but is part of some client class or function that uses the `RestAccess` class.

Processing an element of an archive can be done with two nested `with` contexts. They would work like this:

- An outer `with` statement uses the `zipfile` module to open the archive, creating a `ZipFile` instance.
- An inner `with` statement can open the specific member with the Anscombe quartet CSV file. Inside this context, the application can create a `csv.DictReader` and use the existing `Extract` class to read and process the data.

What's important here is we don't need to unpack the ZIP archive and litter our storage with unzipped files. An application can open and process the elements using the `ZipFile.open()` method.

In addition to downloading the ZIP archive, we may also want to survey the available data sets. For this, a special iterator method is helpful. We'll look at that next.

Getting the data set list

The catalog of data sets is found by using the following path:

<https://www.kaggle.com/api/v1/datasets/list>

The `RestAccess` class can have a `dataset_iter()` method to iterate through the collection of data sets. This is helpful for locating other data sets. It's not required for finding the Anscombe's Quartet, since the `ownerSlug` and `datasetSlug` reference information is already known.

This method can make a `GET` request via the `requests.get()` function to this URL. The response will be on the first page of available Kaggle data sets. The results are provided in pages, and each request needs to provide a page number as a parameter to get subsequent pages.

Each page of results will be a JSON document that contains a sequence of dictionary objects. It has the following kind of structure:

```
[
    {"id": some_number, "ref": "username/dataset", "title": ...},
    {"id": another_number, "ref": "username/dataset", "title": ...},
    etc.
]
```

This kind of two-tiered structure — with pages and items within each page — is the ideal place to use a generator function to iterate through the pages. Within an outer cycle, an inner iteration can yield the individual data set rows from each page.

This nested iteration can look something like the following code snippet:

```
def dataset_iter(url: str, query: dict[str, str]) ->
    Iterator[dict[str, str]]:
    page = 1
    while True:
        response = requests.get(url, params=query | {"page": str(page)})
        if response.status_code == 200:
            details = response.json()
            if details:
                yield from iter(details)
                page += 1
            else:
                break
        elif response.status_code == 429:
            # Too Many Requests
            # Pause and try again processing goes here...
            pass
        else:
            # Unexpected response
            # Error processing goes here...
            break
```

This shows the nested processing of the `while` statement ends when a response contains a page of results with zero entries in it. The processing to handle too many requests is omitted. Similarly, the logging of unexpected responses is also omitted.

A client function would use the `RestAccess` class to scan data sets and would look like the following example:

```
keypath = Path.home()/"Downloads"/"kaggle.json"
with keypath.open() as keyfile:
    credentials = json.load(keyfile)

reader = Access(credentials)
for row in reader.dataset_iter(list_url):
    print(row['title'], row['ref'], row['url'], row['totalBytes'])
```


This will process all of the data set descriptions returned by the `RestReader` object, `reader`. The `dataset_iter()` method needs to accept a `query` parameter that can limit the scope of the search. We encourage you to read the OpenAPI specification to see what options are possible for the `query` parameter. These values will become part of the query string in the HTTP `GET` request.

Here's the formal definition of the interface:

<https://github.com/Kaggle/kaggle-api/blob/master/KaggleSwagger.yaml>

Some of the query parameters include the following:

- The `filetype` query is helpful in locating data in JSON or CSV formats.
- The `maxSize` query can constrain the data sets to a reasonable size. For initial exploration, 1MB is a good upper limit.

Initial spike solutions — without regard to the rate limiting — will turn up at least 80 pages of possible data sets. Handling the rate-limiting response will produce more extensive results, at the cost of some time spent waiting. In the next section, we'll expand this method to handle the error response.

Rate limiting

As with many APIs, the Kaggle API imposes rate-limiting to avoid a **Denial-of-Service (DoS)** attack. For more information see <https://cheatsheetseries.owasp.org/cheatsheets/Denial of Service Cheat Sheet.html>.

Each user has a limited number of requests per second. While the limit is generous for most purposes, it will tend to prevent a simple scan of **all** data sets.

A status code of 429 in a Kaggle response tells the client application that too many requests were made. This "too many requests" error response will have a header with the key `Retry-After`. This header's value is the timeout interval (in seconds) before the next request can be made.

A reliable application will have a structure that handles the 429 vs. 200 responses gracefully. The example in the previous section has a simple `if` statement to check the condition `if response.status_code == 200`. This needs to be expanded to handle these three alternatives:

- A status code of 200 is a good response. If the page has any details, these can be processed; the value of `page` can be incremented. Otherwise, there's no more data making it appropriate to break from the containing `while` statement.
- A status code of 429 means too many requests were made. Get the value of the `Retry-After` and sleep for this period of time.

- Any other status code indicates a problem and should be logged or raised as an exception.

One possible algorithm for return rows and handling rate limiting delays is shown in [Figure 4.3](#).

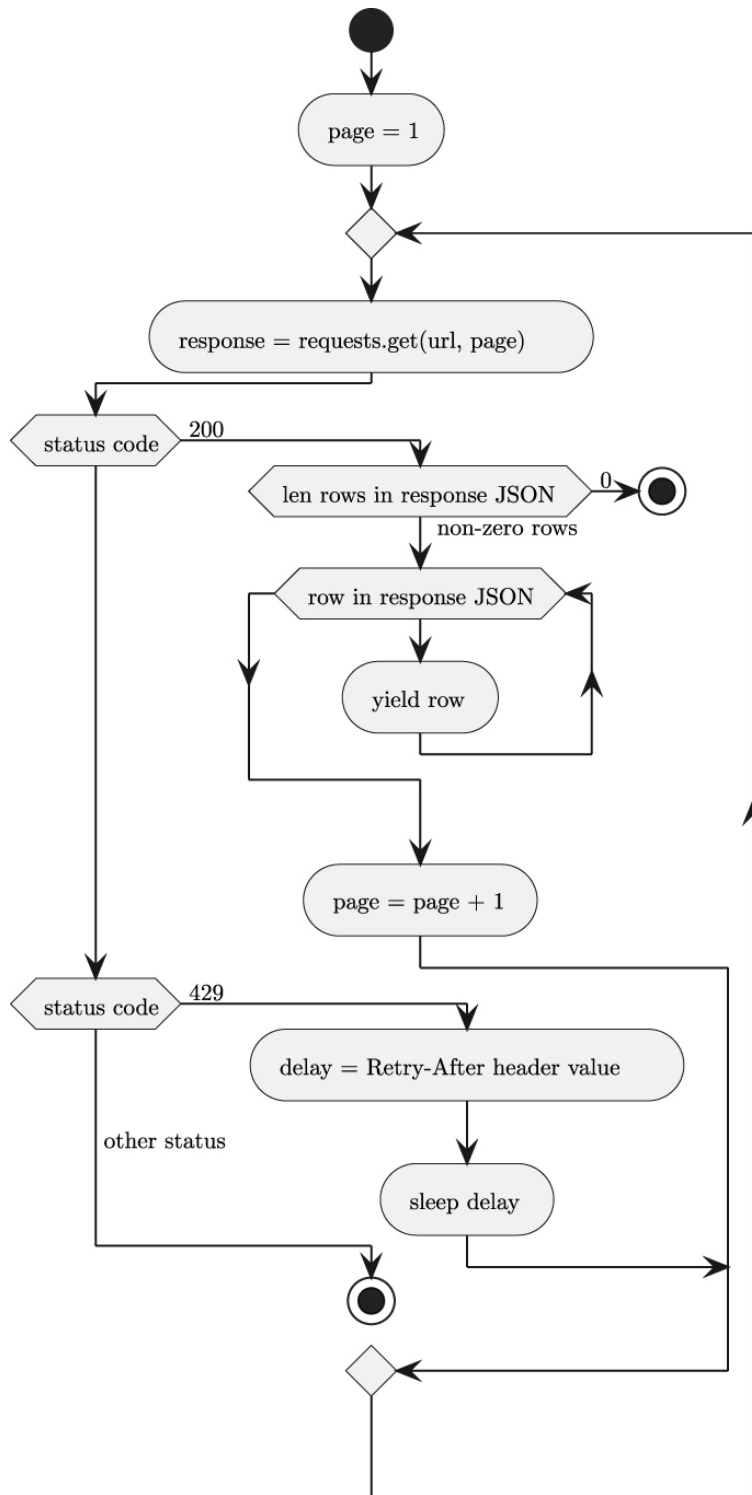


Figure 4.3: Kaggle rate-limited paging

Handling rate limiting will make the application much easier to use. It will also produce more complete results. Using an effective search filter to reduce the number of rows to a sensible level will save a lot of waiting for the retry-after delays.

The `main()` function

The current application design has these distinct features:

1. Extract data from a local CSV file.
2. Download a ZIP archive and extract data from a CSV member of the archive.
3. (Optionally) Survey the data set list to find other interesting data sets to process.

This suggests that our `main()` function should be a container for three distinct functions that implement each separate feature. The `main()` function can parse the command-line arguments, and then make a number of decisions:

- **Local Extract:** If the `-o` option is present (without the `-k` option), then this is a local file extract. This was the solution from an earlier chapter.
- **Download and Extract:** If the `-k` and `-o` options are present, then this will be a download and extract. It will use the `RestAccess` object to get the ZIP archive. Once the archive is opened, the member processing is the solution from an earlier chapter.
- **Survey:** If the `-k` and `-s` (or `--search`) options are present, then this is a search for interesting data sets. You are encouraged to work out the argument design to provide the needed query parameters to the application.
- **Otherwise:** If none of the above patterns match the options, this is incoherent, and an exception should be raised.

Each of these features requires a distinct function. A common alternative design is to use the **Command** pattern and create a class hierarchy with each of the features as a distinct subclass of some parent class.

One central idea is to keep the `main()` function small, and dispatch the detailed work to other functions or objects.

The other central idea is **Don't Repeat Yourself (DRY)**. This principle makes it imperative to **never** copy and paste code between the "Download-and-Extract" feature and the "Local-Extract" feature. The "Download-and-Extract" processing must reuse the "Local-Extract" processing either through subclass inheritance or calling one function from another.

Now that we have a technical approach, it's time to look at the deliverables for this project.

4.1.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- A miniature RESTful web service that provides test responses will be part of the acceptance test.

- Unit tests for the application modules in the `tests` folder.
- Mock objects for the `requests` module will be part of the unit tests.
- Application to download and acquire data from a RESTful web service.

Be sure to include additional packages like `requests` and `beautifulsoup4` in the `pyproject.toml` file. The `pip-compile` command can be used to create a `requirements.txt` usable by the `tox` tool for testing.

We'll look at a few of these deliverables in a little more detail.

Unit tests for the `RestAccess` class

For unit testing, we don't want to involve the `requests` module. Instead, we need to make a mock interface for the `requests` module to confirm the application `RestAccess` module uses the `requests` classes properly.

There are two strategies for plugging in mock objects:

- Implement a dependency injection technique, where the target class is named at run time.
- Use *Monkey Patching* to inject a mock class at test time.

When working with external modules — modules where we don't control the design — monkey patching is often easier than trying to work out a dependency injection technique. When we're building the classes in a module, we often have a need to extend the definitions via subclasses. One of the reasons for creating unique, customized software is to implement change in the unique features of an application rapidly. The non-unique features (RESTful API requests, in this case) change very slowly and don't benefit from flexibility.

We want to create two mock classes, one to replace the `requests.auth.HTTPBasicAuth` class, and one to replace the `requests.get()` function. The mock for the `HTTPBasicAuth` class doesn't do anything; we want to examine the mock object to be sure it was called once with the proper parameters. The mock for the `requests.get()` function needs to create mock `Response` objects for various test scenarios.

We'll need to use the `monkeypatch` fixture of the `pytest` module to replace the real objects with the mock objects for unit testing.

The idea is to create unit tests that have a structure similar to the following example:

```
from unittest.mock import Mock, sentinel, call

def test_rest_access(monkeypatch):
    mock_auth_class = Mock(
        name="Mocked HTTPBasicAuth class",
        return_value=sentinel.AUTH
    )
    monkeypatch.setattr('requests.auth.HTTPBasicAuth', mock_auth_class)
    mock_kaggle_json = {"username": sentinel.USERNAME, "key": sentinel.KEY}
```

```

access = RestAccess(mock_kaggle_json)
assert access.credentials == sentinel.AUTH
assert mock_auth_class.mock_calls == [
    call(sentinel.USERNAME, sentinel.KEY)
]

```

This test case creates a mock for the `HTTPBasicAuth` class. When the class is called to create an instance, it returns a `sentinel` object that can be verified by a test case.

The `monkeypatch` fixture replaces the `requests.auth.HTTPBasicAuth` class with the mock object. After this patch is applied, when the `RestAccess` class initialization attempts to create an instance of the `HTTPBasicAuth` class, it will invoke the mock, and will get a `sentinel` object instead.

The case confirms the `sentinel` object is used by the `RestAccess` instance. The test case also confirms the mock class was called exactly once with the expected values taken from the mocked value loaded from the `kaggle.json` file.

This test case relies on looking inside the `RestAccess` instance. This isn't the best strategy for writing unit tests. A better approach is to provide a mock object for `requests.get()`. The test case should confirm the `requests.get()` is called with a keyword parameter, `auth`, with an argument value of the `sentinel.AUTH` object. The idea of this test strategy is to examine the external interfaces of the `RestAccess` class instead of looking at internal state changes.

Acceptance tests

The acceptance tests need to rely on a *fixture* that mocks the Kaggle web service. The mock will be a process on your local computer, making it easy to stop and start the mock service to test the application. Using an address of `127.0.0.1:8080` instead of `www.kaggle.com` will direct RESTful API requests back to your own computer. The name `localhost:8080` can be used instead of the numeric address `127.0.0.1:8080`. (This address is called the *Loopback Address* because the requests loop back to the same host that created them, allowing testing to proceed without any external network traffic.)

Note that the URL scheme will change to `http:` from `https:`, also. We don't want to implement the full Socket Security Layer (SSL) for acceptance testing. For our purposes, we can trust those components work.

This change to the URLs suggests the application should be designed in such a way to have the `https://www.kaggle.com` portion of each URL provided by a configuration parameter. Then acceptance tests can use `http://127.0.0.1:8080` without having to make any changes to the code.

The mock service must offer a few features of the Kaggle service. The local service needs to respond to `dataset/download` requests properly,

providing a reply with the expected status code and the content with bytes that are a proper ZIP archive.

This mock service will run as a separate application. It will be started (and stopped) by **behave** for a scenario that needs the fixture.

We'll start by looking at the way this service is described in a feature file. This will lead us to look at how to build the mock service. After that, we can look at how this is implemented using **behave** step definitions.

The feature file

The downloading feature is clearly separate from the data acquisition feature. This suggests a new `.feature` file to provide scenarios to describe this feature.

Within this new feature file, we can have scenarios that specifically name the required fixture. A scenario might look like the following example:

```
@fixture.kaggle_server
Scenario: Request for carlmcbriedellis/data-anscombes-quartet
  extracts file from ZIP archive.
  A typical download command might be
  "python src/acquire.py -k kaggle.json -o quartet \
    --zip carlmcbriedellis/data-anscombes-quartet"

  Given proper keys are in "kaggle.json"
  When we run the kaggle download command
  Then log has INFO line with "header: ['mock', 'data']"
  And log has INFO line with "count: 1"
```

The `@fixture.` tag follows the common tagging convention for associating specific fixtures with scenarios. There are many other purposes for tagging scenarios in addition to specifying the fixture to use.

In previous projects, a command to run the application was provided in the `When` step. For this scenario (and many others), the command text became too long to be usefully presented in the Gherkin text. This means the actual command needs to be provided by the function that implements this step.

This scenario's `Then` steps look at the log created by the application to confirm the contents of the file.

A test scenario is part of the overall application's requirements and design.

In the description provided in **Description** there isn't any mention of a log. This kind of gap is common. The test scenario provided additional definitions of the feature omitted from the plain test description.

Some people like to update the documentation to be complete and fully consistent. We encourage flexibility when working on enterprise applications where there are numerous stakeholders. It can be difficult to get everyone's input into the initial presentation or document. Sometimes, re-

quirements appear later in the process when more concrete issues, like expected operating scenarios, are discussed in detail.

The tag information will be used by the **behave** tool. We'll look at how to write a `before_tag()` function to start (and stop) the special mock server for any scenario that needs it.

Before we look at the **behave** integration via a step definition, we'll look at two approaches to testing the client application. The core concept is to create a mock-up of the few elements of the Kaggle API used by the data acquisition application. This mock-up must return responses used by test scenarios. There are two approaches:

- Create a web service application. For acceptance tests, this service must be started and stopped. The acquire application can be configured with a `http://localhost:8080` URL to connect to the test server instead of the Kaggle server. (There are a few common variations on the "localhost" address including `127.0.0.1` and `0.0.0.0`.)
- The other approach is to provide a way to replace the `requests` module with a mocked version of the module. This mocked module returns responses appropriate to the test scenario. This can be done by manipulating the `sys.path` variable to include the directory containing the mocked version of `requests` in front of the `site-packages` directory, which has the real version. It can also be done by providing some application configuration settings that can be replaced with the mocked package.

One approach to creating a complete service that will implement the fixture.

Injecting a mock for the requests package

Replacing the `requests` package requires using dependency injection techniques in the acquire application. A static dependency arises from code like the following:

```
import requests
import requests.auth
```

Later in the module, there may be code like `requests.get(...)` or `requests.auth.HTTPBasicAuth(...)`. The binding to the `requests` module is fixed by both the `import` statement and the references to `requests` and `requests.auth`.

The `importlib` module permits more dynamic binding of modules names, allowing some run-time flexibility. The following, for example, can be used to tailor imports.

```
if __name__ == "__main__":
    # Read from a configuration file
    requests_name = "requests"
    requests = importlib.import_module(requests_name)
    main(sys.argv[1:])
```

The global variable, `requests`, has the imported module assigned to it. This module variable **must** be global; it's an easy requirement to overlook when trying to configure the application for acceptance testing.

Note that the import of the `requests` module (or the mock version) is separated from the remaining `import` statements. This can be the source of some confusion for folks reading this code later, and suitable comments are important for clarifying the way this dependency injection works.

When we looked at unit testing in [*Unit tests for the RestAccess class*](#), we used the **pytest** fixture named `monkeypatch` to properly isolate modules for testing.

Monkey patching isn't a great technique for acceptance testing because the code being tested is not **exactly** the code that will be used. While monkey patching and dependency injection are popular, there are always questions about testing patched software instead of the actual software. In some industries — particularly those where human lives might be at risk from computer-controlled machinery — the presence of a patch for testing may not be allowed. In the next section, we'll look at building a mock service to create and test the acquire application without any patching or changes.

Creating a mock service

A mock service can be built with any web services framework. There are two that are part of the standard library: the `http.server` package and the `wsgiref` package. Either of these can respond to HTTP requests, and can be used to create local services that can mock the Kaggle web service to permit testing our client.

Additionally, any of the well-known web service frameworks can be used to create the mock service. Using a tool like **flask** or **bottle** can make it slightly easier to build a suitable mock service.

To keep the server as simple as possible, we'll use the **bottle** framework. This means adding `bottle==0.12.23` to the `pyproject.toml` file in the `[project.optional-dependencies]` section. This tool is only needed by developers.

The **Bottle** implementation of a RESTful API might look like this:

```
import csv
import io
import json
import zipfile
from bottle import route, run, request, HTTPResponse

@route('/api/v1/datasets/list')
def datasets_list(name):
    # Provide mock JSON documents and Rate throttling

@route('/api/v1/datasets/download/<ownerSlug>/<datasetSlug>')
```



```
def datasets_download(ownerSlug, datasetSlug):
    # Provide mock ZIP archive

if __name__ == "__main__":
    run(host='127.0.0.1', port=8080)
```

While the Kaggle service has numerous paths and methods, this data acquisition project application doesn't use all of them. The mock server only needs to provide routes for the paths the application will actually use.

The `datasets_list` function might include the following example response:

```
@route('/api/v1/datasets/list')
def datasets_list(name):
    page = request.query.page or '1'
    if page == '1':
        mock_body = [
            # Provide attributes as needed by the application under test
            {'title': 'example1'},
            {'title': 'example2'},
        ]
        response = HTTPResponse(
            body=json.dumps(mock_body),
            status=200,
            headers={'Content-Type': 'application/json'})
    else:
        # For error-recovery scenarios, this response may change.
        response = HTTPResponse(
            body=json.dumps([]),
            status=200,
            headers={'Content-Type': 'application/json'})
    return response
```

The `HTTPResponse` object contains the essential features of the responses as seen by the acquisition application's download requests. Each response has content, a status code, and a header that is used to confirm the type of response.

For more comprehensive testing, it makes sense to add another kind of response with the status code of 429 and a header dictionary with `{ 'Retry-After': '30' }`. For this case, the two values of `response` will be more dramatically distinct.

The download needs to provide a mocked ZIP archive. This can be done as shown in the following example:

```
@route('/api/v1/datasets/download/<ownerSlug>/<datasetSlug>')
def datasets_download(ownerSlug, datasetSlug):
    if ownerSlug == "carlmcbrideellis" and datasetSlug == "data-anscombes-quartet":
        zip_content = io.BytesIO()
        with zipfile.ZipFile(zip_content, 'w') as archive:
            target_path = zipfile.Path(archive, 'Anscombe_quartet_data.csv')
            with target_path.open('w') as member_file:
                writer = csv.writer(member_file)
```

```

        writer.writerow(['mock', 'data'])
        writer.writerow(['line', 'two'])
    response = HTTPResponse(
        body=zip_content.getvalue(),
        status=200,
        headers={"Content-Type": "application/zip"}
    )
    return response
# All other requests...
response = HTTPResponse(
    status=404
)
return response

```

This function will only respond to one specifically requested combination of `ownerSlug` and `datasetSlug`. Other combinations will get a 404 response, the status code for a resource that can't be found.

The `io.BytesIO` object is an in-memory buffer that can be processed like a file. It is used by the `zipfile.ZipFile` class to create a ZIP archive. A single member is written to this archive. The member has a header row and a single row of data, making it easy to describe in a Gherkin scenario. The response is built from the bytes in this file, a status code of 200, and a header telling the client the content is a ZIP archive.

This service can be run on the desktop. You can use a browser to interact with this server and confirm it works well enough to test our application.

Now that we've seen the mock service that stands in for Kaggle.com, we can look at how to make the **behave** tool run this service when testing a specific scenario.

Behave fixture

We've added a `fixture.kaggle_server` to the scenario. There are two steps to make this tag start the server process running for a given scenario. These steps are:

1. Define a generator function. This will start a subprocess, yield something, and then kill the subprocess.
2. Define a `before_tag()` function to inject the generator function into the step processing.

Here's a generator function that will update the context, and start the mock Kaggle service.

```

from collections.abc import Iterator
from typing import Any
import subprocess
import time
import os
import sys
from behave import fixture, use_fixture
from behave.runner import Context

@fixture
def kaggle_server(context: Context) -> Iterator[Any]:

```

```

if "environment" not in context:
    context.environment = os.environ
context.environment["ACQUIRE_BASE_URL"] = "http://127.0.0.1:8080"
# Save server-side log for debugging
server = subprocess.Popen(
    [sys.executable, "tests/mock_kaggle_bottle.py"],
)
time.sleep(0.5) # 500 ms delay to allow the service to open a socket
yield server
server.kill()

```

The portion of the function before the `yield` statement is used during the scenario setup. This will add value to the context that will be used to start the application under test. After the yielded value has been consumed by the **Behave** runner, the scenario executes. When the scenario is finished, one more value is requested from this generator; this request will execute the statements after the `yield` statement. There's no subsequent `yield` statement; the `StopIteration` is the expected behavior of this function.

This `kaggle_server()` function must be used in a scenario when the `@fixture` tag is present. The following function will do this:

```

from behave import use_fixture
from behave.runner import Context

def before_tag(context: Context, tag: str) -> None:
    if tag == "fixture.kaggle_server":
        # This will invoke the definition generator.
        # It consumes a value before and after the tagged scenario.
        use_fixture(kaggle_server, context)

```

When the `@fixture.kaggle_server` tag is present, this function will inject the `kaggle_server()` generator function into the overall flow of processing by the runner. The runner will make appropriate requests of the `kaggle_server()` generator function to start and stop the service.

These two functions are placed into the `environment.py` module where the **behave** tool can find and use them.

Now that we have an acceptance test suite, we can turn to implement the required features of the `acquire` application.

Kaggle access module and refactored main application

The goal, of course, is two-fold:

- Add a `kaggle_client.py` module. The unit tests will confirm this works.
- Rewrite the `acquire.py` module from [*Chapter 3, Project 1.1: Data Acquisition Base Application*](#) to add the download feature.

The [*Approach*](#) section provides some design guidance for building the application. Additionally, the previous chapter, [*Chapter 3, Project 1.1: Data Acquisition Base Application*](#) provides the baseline application into which the new acquisition features should be added.

The acceptance tests will confirm the application works correctly.

Given this extended capability, you are encouraged to hunt for additional, interesting data sets. The new application can be revised and extended to acquire new, interesting data in other formats.

Now that we have data acquired from the web in a tidy, easy-to-use format, we can look at acquiring data that isn't quite so tidy. In the next section, we'll look at how to scrape data out of an HTML page.

4.2 Project 1.3: Scrape data from a web page

In some cases, we want data that's provided by a website that doesn't have a tidy API. The data is available via an HTML page. This means the data is surrounded by HTML *markup*, text that describes the semantics or structure of the data.

We'll start with a description of the application, and then move on to talk about the architectural approach. This will be followed with a detailed list of deliverables.

4.2.1 Description

We'll continue to describe projects designed to acquire some data for further analysis. In this case, we'll look at data that is available from a website, but is embedded into the surrounding HTML markup. We'll continue to focus on Anscombe's Quartet data set because it's small and diagnosing problems is relatively simple. A larger data set introduces additional problems with time and storage.

Parts of this application are an extension to the project in [*Project 1.2: Acquire data from a web service*](#). The essential behavior of this application will be similar to the previous project. This project will use a CLI application to grab data from a source.

The User Experience (UX) will also be a command-line application with options to fine-tune the data being gathered. Our expected command line should like something like the following:

```
% python src/acquire.py -o quartet --page "https://en.wikipedia.org/wiki/Anscombe's_quartet" --cap
```

The `-o quartet` argument specifies a directory into which four results are written. These will have names like `quartet/series_1.json`.

The table is buried in the HTML of the URL given by the `--page` argument. Within this HTML, the target table has a unique `<caption>` tag: `<caption>Anscombe's quartet</caption>`.

4.2.2 About the source data

This data embedded in HTML markup is generally marked up with the `<table>` tag. A table will often have the following markup:

```
<table class="wikitable" style="text-align: center; margin-left:auto;
margin-right:auto;" border="1">
  <caption>Anscombe's quartet</caption>
  <tbody>
    <tr>
      <th colspan="2">I</th>
      etc.
    </tr>
    <tr>
      <td><i>x</i></td>
      <td><i>y</i></td>
      etc.
    </tr>
    <tr>
      <td>10.0</td>
      <td>8.04</td>
      etc.
    </tr>
  </tbody>
</table>
```

In this example, the overall `<table>` tag will have two child tags, a `<caption>` and a `<tbody>`.

The table's body, within `<tbody>`, has a number of rows wrapped in `<tr>` tags. The first row has headings in `<th>` tags. The second row also has headings, but they use the `<td>` tags. The remaining rows have data, also in `<td>` tags.

This structure has a great deal of regularity, making it possible to use a parser like **Beautiful Soup** to locate the content.

The output will match the extraction processing done for the previous projects. See [Chapter 3, Project 1.1: Data Acquisition Base Application](#), for the essence of the data acquisition application.

This section has looked at the input and processing for this application. The output will match earlier projects. In the next section, we'll look at the overall architecture of the software.

4.2.3 Approach

We'll take some guidance from the C4 model (<https://c4model.com>) when looking at our approach:

- **Context:** For this project, a context diagram would show a user extracting data from a source. You may find it helpful to draw this diagram.
- **Containers:** One container is the user's personal computer. The other container is the Wikipedia website, which provides the data.
- **Components:** We'll address the components below.
- **Code:** We'll touch on this to provide some suggested directions.

It's important to consider this application as an extension to the project in [Chapter 3, Project 1.1: Data Acquisition Base Application](#). The base level of architectural design is provided in that chapter.

In this project, we'll be adding a new `html_extract` module to capture and parse the data. The overall application in the `acquire` module will change to use the new features. The other modules should remain unchanged.

A new architecture that handles the download of HTML data and the extraction of a table from the source data is shown in **Figure 4.4**.

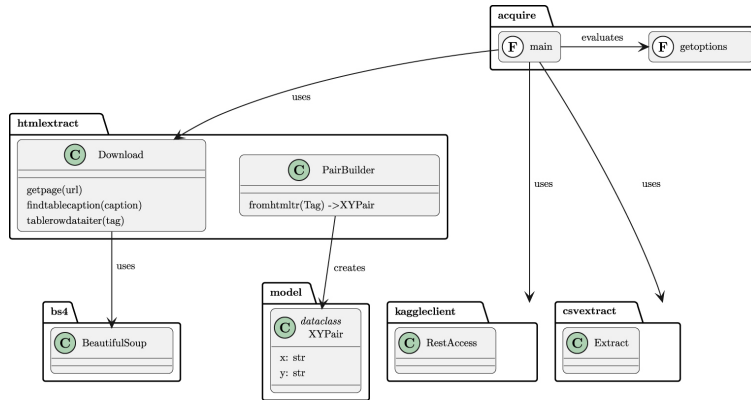


Figure 4.4: Revised Component Design

This diagram suggested classes for the new `html_extract` module. The `Download` class uses `urllib.request` to open the given URL and read the contents. It also uses the `bs4` module (**Beautiful Soup**) to parse the HTML, locate the table with the desired caption, and extract the body of the table.

The `PairBuilder` class hierarchy has four implementations, each appropriate for one of the four data series. Looking back at **Chapter 3, Project 1.1: Data Acquisition Base Application**, there's a profound difference between the table of data shown on the Wikipedia page, and the CSV source file shown in that earlier project. This difference in data organization requires slightly different pair-building functions.

Making an HTML request with `urllib.request`

The process of reading a web page is directly supported by the `urllib.request` module. The `url_open()` function will perform a GET request for a given URL. The return value is a file-like object — with a `read()` method — that can be used to acquire the content.

This is considerably simpler than making a general RESTful API request where there are a variety of pieces of information to be uploaded and a variety of kinds of results that might be downloaded. When working with common GET requests, this standard library module handles the ordinary processing elegantly.

A suggested design for the first step in the operation is the following function:

```
from urllib.request import urlopen
from bs4 import BeautifulSoup, Tag
```

```
def get_page(url: str) -> BeautifulSoup:
    return BeautifulSoup(
        urlopen(url), "html.parser"
    )
```

The `urlopen()` function will open the URL as a file-like object, and provide that file to the `BeautifulSoup` class to parse the resulting HTML.

A `try:` statement to handle potential problems is not shown. There are innumerable potential issues when reaching out to a web service, and trying to parse the resulting content. You are encouraged to add some simple error reporting.

In the next section, we'll look at extracting the relevant table from the parsed HTML.

HTML scraping and BeautifulSoup

The BeautifulSoup data structure has a `find_all()` method to traverse the structure. This will look for tags with specific kinds of properties. This can examine the tag, the attributes, and even the text content of the tag.

See <https://www.crummy.com/software/BeautifulSoup/bs4/doc/#find-all>.

In this case, we need to find a `<table>` tag with a `caption` tag embedded within it. That `caption` tag must have the desired text. This search leads to a bit more complex investigation of the structure. The following function can locate the desired table.

```
def find_table_caption(
    soup: BeautifulSoup,
    caption_text: str = "Anscombe's quartet"
) -> Tag:
    for table in soup.find_all('table'):
        if table.caption:
            if table.caption.text.strip() == caption_text.strip():
                return table
    raise RuntimeError(f"<table> with caption {caption_text!r} not found")
```

Some of the tables lack captions. This means the expression `table.caption.text` won't work for string comparison because it may have a `None` value for `table.caption`. This leads to a nested cascade of `if` statements to be sure there's a `<caption>` tag before checking the text value of the tag.

The `strip()` functions are used to remove leading and trailing whitespace from the text because blocks of text in HTML can be surrounded by whitespace that's not displayed, making it surprising when it surfaces as part of the content. Stripping the leading and trailing whitespace makes it easier to match.

The rest of the processing is left for you to design. This processing involves finding all of the `<tr>` tags, representing rows of the table. Within each row (except the first) there will be a sequence of `<td>` tags representing the cell values within the row.

Once the text has been extracted, it's very similar to the results from a `csv.reader`.

After considering the technical approach, it's time to look at the deliverables for this project.

4.2.4 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- Unit tests for the application modules in the `tests` folder.
- Mock HTML pages for unit testing will be part of the unit tests.
- Application to acquire data from an HTML page.

We'll look at a few of these deliverables in a little more detail.

Unit test for the `html_extract` module

The `urlopen()` function supports the `http:` and `https:` schemes. It also supports the `file:` protocol. This allows a test case to use a URL of the form `file:///path/to/a/file.html` to read a local HTML file. This facilitates testing by avoiding the complications of accessing data over the internet.

For testing, it makes sense to prepare files with the expected HTML structure, as well as invalid structures. With some local files as examples, a developer can run test cases quickly.

Generally, it's considered a best practice to mock the `BeautifulSoup` class. A fixture would respond to the various `find_all()` requests with mock tag objects.

When working with HTML, however, it seems better to provide mock HTML. The wide variety of HTML seen in the wild suggests that time spent with real HTML is immensely valuable for debugging.

Creating `BeautifulSoup` objects means the unit testing is more like integration testing. The benefits of being able to test a wide variety of odd and unusual HTML seems to be more valuable than the cost of breaking the ideal context for a unit test.

Having example HTML files plays well with the way **pytest** fixtures work. A fixture can create a file and return the path to the file in the form of a URL. After the test, the fixture can remove the file.

A fixture with a test HTML page might look like this:

```
from pytest import fixture
from textwrap import dedent

@fixture
def example_1(tmp_path):
```



```

html_page = tmp_path / "works.html"
html_page.write_text(
    dedent("""\
<!DOCTYPE html>
<html>
etc.
</html>
""")
)
yield f"file://{str(html_page)}"
html_page.unlink()

```

This fixture uses the `tmp_path` fixture to provide access to a temporary directory used only for this test. The file, `works.html`, is created, and filled with an HTML page. The test case should include multiple `<table>` tags, only one of which was the expected `<caption>` tag.

The `dedent()` function is a handy way to provide a long string that matches the prevailing Python indent. The function removes the indenting whitespace from each line; the resulting text object is not indented.

The return value from this fixture is a URL that can be used by the `urlopen()` function to open and read this file. After the test is completed, the final step (after the `yield` statement) will remove the file.

A test case might look something like the following:

```

def test_steps(example_1):
    soup = html_extract.get_page(example_1)
    table_tag = html_extract.find_table_caption(soup, "Anscombe's quartet")
    rows = list(html_extract.table_row_data_iter(table_tag))
    assert rows == [
        [],
        ['Keep this', 'Data'],
        ['And this', 'Data'],
    ]

```

The test case uses the `example_1` fixture to create a file and return a URL referring to the file. The URL is provided to a function being tested. The functions within the `html_extract` module are used to parse the HTML, locate the target table, and extract the individual rows.

The return value tells us the functions work properly together to locate and extract data. You are encouraged to work out the necessary HTML for good — and bad — examples.

Acceptance tests

As noted above in *[Unit test for the html extract module](#)*, the acceptance test case HTML pages can be local files. A scenario can provide a local `file://` URL to the application and confirm the output includes properly parsed data.

The Gherkin language permits including large blocks of text as part of a scenario.

We can imagine writing the following kinds of scenarios in a feature file:

```
Scenario: Finds captioned table and extracts data
  Given an HTML page "example_1.html"
  """
    <!DOCTYPE html>
    <html>
      etc. with multiple tables.
    </html>
  """
  When we run the html extract command
  Then log has INFO line with "header: ['Keep this', 'Data']"
  And log has INFO line with "count: 1"
```

The HTML extract command is quite long. The content is available as the `context.text` parameter of the step definition function. Here's what the step definition for this given step looks like:

```
from textwrap import dedent

@given(u'an HTML page "{filename}")
def step_impl(context, filename):
    context.path = Path(filename)
    context.path.write_text(dedent(context.text))
    context.add_cleanup(context.path.unlink)
```

The step definition puts the path into the context and then writes the HTML page to the given path. The `dedent()` function removes any leading spaces that may have been left in place by the **behave** tool. Since the path information is available in the context, it can be used by the **When** step. The `context.add_cleanup()` function will add a function that can be used to clean up the file when the scenario is finished. An alternative is to use the environment module's `after_scenario()` function to clean up.

This scenario requires an actual path name for the supplied HTML page to be injected into the text. For this to work out well, the step definition needs to build a command from pieces. Here's one approach:

```
@when(u'we run the html extract command')
def step_impl(context):
    command = [
        'python', 'src/acquire.py',
        '-o', 'quartet',
        '--page', '$URL',
        '--caption', "Anscombe's quartet"
    ]
    url = f"file://{str(context.path.absolute())}"
    command[command.index('$URL')] = url
    print(shlex.join(command))
    # etc. with subprocess.run() to execute the command
```

In this example, the command is broken down into individual parameter strings. One of the strings must be replaced with the actual file name. This works out nicely because the `subprocess.run()` function works well with a parsed shell command. The `shlex.split()` function can be

used to decompose a line, honoring the complex quoting rules of the shell, into individual parameter strings.

Now that we have an acceptance test suite, we may find the `acquire` application doesn't pass all of the tests. It's helpful to define done via an acceptance test and then develop the required HTML extract module and refactor the main application. We'll look at these two components next.

HTML extract module and refactored main application

The goal for this project is two-fold:

- Add an `html_extract.py` module. The unit tests will confirm this module works.
- Rewrite the `acquire.py` module from [*Chapter 3, Project 1.1: Data Acquisition Base Application*](#) to add the HTML download and extract the feature.

The [*Approach*](#) section provides some design guidance for building the application. Additionally, the previous chapter, [*Chapter 3, Project 1.1: Data Acquisition Base Application*](#), provides the baseline application into which the new acquisition features should be added.

The acceptance tests will confirm the application works correctly to gather data from the Kaggle API.

Given this extended capability, you can hunt for data sets that are presented in web pages. Because of the consistency of Wikipedia, it is a good source of data. Many other sites provide relatively consistent HTML tables with interesting data.

In these two projects, we've extended our ability to acquire data from a wide variety of sources.

4.3 Summary

This chapter's projects have shown examples of the following features of a data acquisition application:

- Web API integration via the **requests** package. We've used the Kaggle API as an example of a RESTful API that provides data for download and analysis.
- Parsing an HTML web page using the **Beautiful Soup** package.
- Adding features to an existing application and extending the test suite to cover these new alternative data sources.

A challenging part of both of these projects is creating a suite of acceptance tests to describe the proper behavior. Pragmatically, a program without automated tests cannot be trusted. The tests are every bit as important as the code they're exercising.

In some enterprises, the definition of done is breezy and informal. There may be a presentation or an internal memo or a whitepaper that de-

scribes the desired software. Formalizing these concepts into tangible test cases is often a significant effort. Achieving agreements can become a source of turmoil as stakeholders slowly refine their understanding of how the software will behave.

Creating mock web services is fraught with difficulty. Some API's permit downloading an `openapi.json` file with the definition of the API complete with examples. Having concrete examples, provided by the host of the API, makes it much easier to create a mock service. A mock server can load the JSON specification, navigate to the example, and provide the official response.

Lacking an OpenAPI specification with examples, developers need to write spike solutions that download detailed responses. These responses can then be used to build mock objects. You are strongly encouraged to write side-bar applications to explore the Kaggle API to see how it works.

In the next chapter, we'll continue this data extraction journey to include extracting data from SQL databases. Once we've acquired data, we'll want to inspect it. [*Chapter 6, Project 2.1: Data Inspection Notebook*](#), will introduce an inspection step.

4.4 Extras

Here are some ideas for you to add to these projects.

4.4.1 Locate more JSON-format data

A search of Kaggle will turn up some other interesting data sets in JSON format.

- <https://www.kaggle.com/datasets/rtatman/iris-dataset-json-version>: This data set is famous and available in a number of distinct formats.
- <https://www.kaggle.com/datasets/conoor/stack-overflow-tags-usage>
- <https://www.kaggle.com/datasets/queyrusi/the-warship-dataset>

One of these is a JSON download. The other two are ZIP archives that contain JSON-format content.

This will require revising the application's architecture to extract the JSON format data instead of CSV format data.

An interesting complication here is the distinction between CSV data and JSON data:

- CSV data is pure text, and later conversions are required to make useful Python objects.
- Some JSON data is converted to Python objects by the parser. Some data (like timestamps) will be left as text.

At acquisition time, this doesn't have a significant impact. However, when we get to [*Chapter 9, Project 3.1: Data Cleaning Base*](#)

Application. we'll have to account for data in a text-only form distinct from data with some conversions applied.

The Iris data set is quite famous. You can expand on the designs in this chapter to acquire Iris data from a variety of sources. The following steps could be followed:

- 1. Start with the Kaggle data set in JSON format. Build the needed model, and extract modules to work with this format.
- 2. Locate other versions of this data set in other formats. Build the needed extract modules to work with these alternative formats.

Once a core acquisition project is complete, you can leverage this other famous data set as an implementation choice for later projects.

4.4.2 Other data sets to extract

See the CO2 PPM — Trends in Atmospheric Carbon Dioxide data set, available at <https://datahub.io/core/co2-ppm>, for some data that is somewhat larger. This page has a link to an HTML table with the data.

See <https://datahub.io/core/co2-ppm/r/0.html> for a page with the complete data set as an HTML table. This data set is larger and more complicated than Anscombe's Quartet. In **Chapter 6, Project 2.1: Data Inspection Notebook**, we'll address some of the special cases in this data set.

4.4.3 Handling schema variations

The two projects in this chapter each reflect a distinct schema for the source data.

One CSV format can be depicted via an **Entity-Relationship Diagram (ERD)**, shown in **Figure 4.5**.

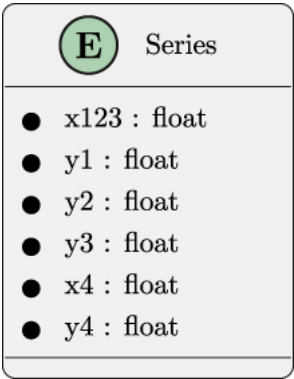


Figure 4.5: Source entity-relationship diagram

One column, `x_123`, is the x-value of three distinct series. Another column, `x_4`, is the x-value for one series.

A depiction of the HTML format as an ERD is shown in **Figure 4.6**.

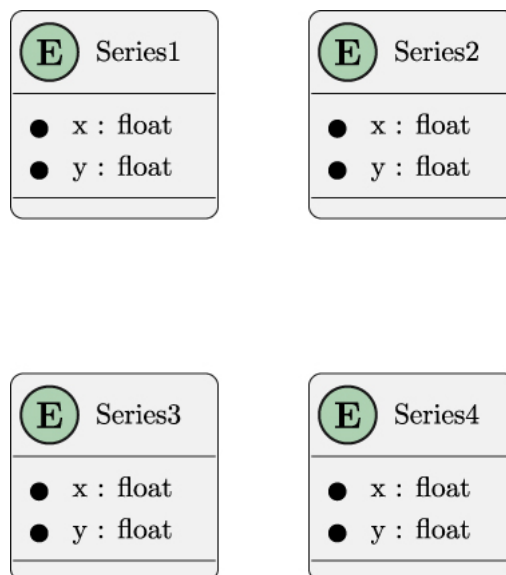


Figure 4.6: Notional entity-relationship diagram

The x-values are repeated as needed.

This difference requires several distinct approaches to extracting the source data.

In these projects, we've implemented this distinction as distinct subclasses of a `PairBuilder` superclass.

An alternative design is to create distinct functions with a common type signature:

```
PairBuilder: TypeVar = Callable[[list[str]], XYPair]
```

Making each conversion a function eliminates the overhead of the class definition.

This rewrite can be a large simplification. It will not change any acceptance tests. It will, however, require numerous changes to unit tests.

The functional design offers some simplification over class-based design. You are encouraged to perform the functional redesign of the suggestions in this book.

4.4.4 CLI enhancements

The CLI for these two projects is left wide open, permitting a great deal of design flexibility and alternatives. Because the CLI is part of externally visible behavior, it becomes necessary to write acceptance tests for the various CLI options and arguments.

As noted in [*Additional acceptance scenarios*](#), there are a number of acceptance test scenarios that are not on the "happy path" where the application works. These additional scenarios serve to catalog a number of erroneous uses of the application.

This becomes more important as more features are added and the CLI becomes more complicated. You are encouraged to write acceptance tests for invalid CLI use.

4.4.5 Logging

Logging is an important part of data acquisition. There are a number of potential problems exposed by these two projects. A website might be unresponsive, or the API may have changed. The HTML may have been reformatted in some subtle way.

A *debug* or *verbose* mode should be available to expose the interactions with external services to be sure of the HTTP status codes and headers.

Additionally, count values should be displayed to summarize the bytes downloaded, the lines of text examined, and the number of `XYPair` objects created. The idea is to characterize the inputs, the various processing steps, and the outputs.

These counts are essential for confirming that data is processed and filtered correctly. They're an important tool for making parts of the processing more observable. A user wants to confirm that all of the downloaded data is either part of the results or filtered and discarded for a good reason.

You are encouraged to include counts for input, processing, and output in the log.